Leçon 6 : Pratique du langage SQL avec SQLite

Dr SANGARE Seydou

Table des matières

Objectifs 3

Introduction 4

I - Utilisation de DB Browser 5

II - Langage de Définition de Données (LDD - Data Definition Language) 6

III - Langage de Modification de Données (LMD - Data Manipulation Language) 8

IV - Langage d'Interrogation de Données (LID - Data Query Language) 12

V - Langage de Contrôle de Données (LCD - Data Control Language) 17

Conclusion 19

Objectifs

A la fin de ce cours, l'apprenant sera capable de :

- Comprendre la syntaxe de base des requêtes SQL ;
- Ecrire les requêtes SQL pour créer, lire, mettre à jour et supprimer des données ;
- Utiliser des clauses et des fonctions SQL pour filtrer et manipuler les résultats ;
- Appliquer des concepts de normalisation lors de la création de tables ;
- Contrôler les accès.

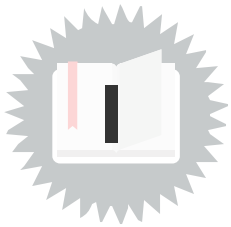
Introduction

SQL (Structured Query Language) est le langage universel pour communiquer avec les bases de données relationnelles. Que vous soyez data analyst, développeur web ou simplement curieux, maîtriser SQL est une compétence incontournable.

SQLite est un moteur de base de données léger, rapide et sans serveur, parfait pour l'apprentissage, le prototypage et même de nombreuses applications en production. Toutes les commandes que nous allons voir sont standard et fonctionnent sur la plupart des autres systèmes (MySQL, PostgreSQL, etc.), avec de légères variations.

Nous allons explorer les concepts fondamentaux du langage SQL, qui est essentiel pour interagir avec les bases de données relationnelles. Nous aborderons des sujets allant de la définition des données à la manipulation des données, en passant par le contrôle d'accès. À la fin de cette leçon, vous serez capable d'écrire des requêtes SQL pour créer, lire, mettre à jour et supprimer des données dans une base de données SQLite.

Utilisation de DB Browser



Étape 1 : Création d'une base de données pour la boutique en ligne

1. Cliquez sur **"Nouvelle base de données"**.
2. Choisissez un emplacement et un nom (ex : boutique.db).
3. DB Browser ouvre une interface pour créer la première table.

Étape 2: Création des tables essentielles

Pour une boutique, on commencera par créer des tables clés :

- **Produits** (id, nom, description, prix, stock)
- **Clients** (id, nom, email, téléphone)
- **Commandes** (id, client_id, date, total)

Procédure :

1. Dans l'onglet **Structure**, cliquez sur **"Créer une table"**.
2. Ajoutez un nom à la table (ex : Produits).
3. Ajoutez les champs un par un avec leurs types (INTEGER, TEXT, REAL).
4. Définissez une clé primaire (ex : id, INTEGER, autoincrement).
5. Cliquez sur **OK** puis sur **Écrire les modifications** pour sauvegarder.

Étape 3: Ajouter des données

1. Allez dans l'onglet **Parcourir les données**.
2. Sélectionnez la table (ex : Produits).
3. Cliquez sur **Ajouter un enregistrement**.
4. Remplissez les champs (nom, description, prix, stock).
5. Validez pour insérer la ligne.

Étape 4: Pour exécuter les requêtes SQL

1. Passez à l'onglet **Exécuter SQL**.
2. Tapez une requête, par exemple :

Langage de Définition de Données (LDD - Data Definition Language)



Le LDD regroupe les commandes qui permettent de **définir et modifier la structure** de la base de données : les tables, les colonnes, les types de données, les clés.

Commandes clés : `CREATE`, `ALTER`, `DROP`

1. Création d'une base et d'une table (CREATE)

Créons notre base `Boutique` et notre première table `Article`.

La relation **Article(*id*, nom, description, prix, stock, id_categorie)**

`Schéma de la relation` **`Article(id:entier, nom:chaîne, description:chaîne, prix:entier, stock:entier, id_categorie:entier)`**

- ***id*** : identifiant de l'article (type entier)
- **Nom** : Nom de l'article (type chaîne de caractères)
- **description** : description de l'article (type chaîne de caractères)
- **prix** : prix de l'article (type entier)
- **stock** : stock de l'article (type entier)
- **id_categorie** : clé étrangère qui est l'identifiant de la table catégorie (type entier)

```sql
-- SQLite créera la base automatiquement au premier connexion.
-- Création de la table 'article'
CREATE TABLE article (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nom TEXT NOT NULL,
    description TEXT,
    prix REAL NOT NULL CHECK (prix >= 0),
    stock INTEGER DEFAULT 0,
    id_categorie INTEGER,
    FOREIGN KEY (id_categorie) REFERENCES categorie(id)
);
```

Image 1 Requête SQL pour la création de la table article

Explications :

- `CREATE TABLE` : Commande pour créer une table.
- `id INTEGER PRIMARY KEY AUTOINCREMENT` : Une clé primaire qui s'incrémente automatiquement. C'est un identifiant unique.
- `TEXT`, `REAL`, `INTEGER` : Types de données en SQLite.
- `NOT NULL` : Contrainte qui empêche la valeur d'être nulle.
- `CHECK (prix >= 0)` : Contrainte qui s'assure que le prix n'est pas négatif.
- `DEFAULT 0` : Valeur par défaut si aucune n'est spécifiée.

- `FOREIGN KEY` : Clé étrangère qui crée un lien vers une autre table (que nous allons créer)

Créons la table `categorie` référencée :

La relation **`categorie`(id_categorie, nom_categorie)**

Schéma de la relation **`categorie`(id_categorie *:entier*, nom_categorie :chaîne)**

- **id_categorie** : identifiant de la catégorie (type entier)
- **nom_categorie** : Nom de la catégorie (type chaîne de caractères)

```sql
CREATE TABLE categorie (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nom TEXT NOT NULL UNIQUE
);
```

Image 2 Requête SQL pour la création de la table catégorie

2. Modification d'une table (ALTER)

Imaginons que nous voulons ajouter une colonne fournisseur à la table Article.

```sql
ALTER TABLE article ADD COLUMN fournisseur TEXT;
```

Image 3 Requête SQL pour modifier la table Article en lui ajoutant une nouvelle colonne

3. Suppression (DROP)

Attention ! Cette commande est destructive et supprime définitivement la table et ses données.

```sql
DROP TABLE article; -- A utiliser avec extrême prudence !
```

Image 4 Commande SQL pour supprimer une table

Langage de Modification de Données (LMD - Data Manipulation Language)



Le LMD permet de **modifier le contenu** des tables : ajouter, modifier, supprimer des lignes.

Commandes clés : `INSERT`, `UPDATE`, `DELETE`

1. Ajout de données (INSERT)

Ajoutons une catégorie et un article.

```
INSERT INTO categorie (nom) VALUES ('Livre');
INSERT INTO categorie (nom) VALUES ('Vêtement');

INSERT INTO article (nom, description, prix, stock, id_categorie)
VALUES ('Apprendre SQL', 'Un livre pour débutants', 29.99, 10, 1);
```

Image 2 Exemple de requête pour insérer des données dans les tables

Ces trois lignes de code signifient :

1. **"Ajoute une nouvelle catégorie appelée 'Livre' dans la table `categorie`."**
2. **"Ajoute une nouvelle catégorie appelée 'Vêtement' dans la table `categorie`."**
3. **"Ajoute un nouvel article appelé 'Apprendre SQL' dans la table `article`, avec sa description, son prix, son stock, et fais-le appartenir à la catégorie qui a l'id 1 (la catégorie 'Livre')."**

En résumé, on **insère** d'abord les catégories, puis on **insère** un article qui est lié à l'une de ces catégories.

Requête 1 & 2 : Insertion dans la table `categorie`

- **`INSERT INTO`** : La commande qui signifie "Insérer dans". C'est le verbe de l'action.
- **`categorie`** : Le nom de la table **dans laquelle** on veut insérer des données.
- **`(nom)`** : La liste des **colonnes** dans lesquelles on va spécifier des valeurs. Ici, on ne spécifie que la colonne `nom`. Les autres colonnes (comme `id`) seront remplies automatiquement.
- **`VALUES`** : Le mot-clé qui introduit les **valeurs** à insérer.
- **`('Livre')`** : La ou les valeurs à insérer, **dans le même ordre** que la liste des colonnes. Ici, on insère la valeur `'Livre'` (une chaîne de caractères) dans la colonne `nom`.

Résultat : Après ces deux insertions, la table `categorie` ressemble à ceci. Notez que l'`id` est généré automatiquement.

id	nom
1	Livre
2	Vêtement

Requête 3 : Insertion dans la table `article`

- **`INSERT INTO article`** : "Insère dans la table `article`".
- **`(nom, description, prix, stock, id_categorie)`** : La liste **explicite** des colonnes que l'on va remplir.
- **`VALUES (...)`** : Les valeurs correspondant à chaque colonne listée, **dans le même ordre**.
    - `'Apprendre SQL'` → va dans la colonne `nom`
    - `'Un livre pour débutants'` → va dans la colonne `description`
    - `29.99` → va dans la colonne `prix` (c'est un nombre réel, donc pas de quotes)
    - `10` → va dans la colonne `stock` (c'est un entier, donc pas de quotes)
    - `1` → va dans la colonne `id_categorie` (c'est l'`id` de la catégorie "Livre" insérée précédemment)

Résultat : Après cette insertion, la table `article` contient une nouvelle ligne. L'`id` est généré automatiquement, et la clé étrangère `id_categorie` crée un lien vers la table `categorie`.

id	nom	description	prix	stock	id_categorie
1	Apprendre SQL	Un livre pour débutants	29.99	10	1

L'Importance de l'Ordre et des Relations:

1. **Ordre Logique :** On **doit** d'abord insérer la catégorie ("Livre") pour qu'elle existe et obtienne un `id` (ici, `1`).
2. **Lien Relationnel :** Ensuite, quand on insère l'article, on peut utiliser cet `id` (`1`) pour la colonne `id_categorie`. Cela crée un lien solide entre l'article "Apprendre SQL" et la catégorie "Livre".
3. **Intégrité des Données :** Si on avait essayé d'insérer l'article avec `id_categorie = 99` alors qu'aucune catégorie avec cet ID n'existe, SQLite aurait renvoyé une erreur (si la contrainte de clé étrangère est activée), protégeant ainsi la cohérence de vos données.

Bonnes Pratiques Illustrées

- **Liste Explicite des Colonnes :** Utiliser `INSERT INTO table (col1, col2)...` au lieu de juste `INSERT INTO table...` est plus sûr et plus clair.
- **Gestion des `ID` :** On ne spécifie jamais la valeur pour une colonne `AUTOINCREMENT` (comme l'`id`), le moteur de base de données s'en charge.
- **Relations :** On fait toujours référence à d'autres tables via leur **clé primaire** (`id`), jamais par leur nom textuel (on écrit `id_categorie = 1`, pas `categorie = 'Livre'`).

En résumé : Ces requêtes `INSERT INTO` sont la base de **l'ajout de données** dans votre base. Elles permettent de peupler les tables et d'établir des liens entre elles, jetant ainsi les bases pour des requêtes de sélection (`SELECT`) complexes et riches de sens.

2. Mise à jour de données (UPDATE)

Le livre "Apprendre SQL" est en promotion.

```sql
UPDATE article
SET prix = 24.99, stock = stock - 1 -- On décrémente aussi le stock
WHERE nom = 'Apprendre SQL'; -- La clause WHERE est CRUCIALE pour ne pas tout modifier !
```

Image 3 Requête de mise à jour

La requête `UPDATE article SET prix = 24.99, stock = stock - 1 WHERE nom = 'Apprendre SQL';` signifie :

"Trouve dans la table 'article' tous les articles dont le nom est exactement 'Apprendre SQL'. Pour ces articles-là, et uniquement pour eux, change leur prix pour 24,99 € et décrémente leur stock de 1 unité."

En résumé, elle permet de **mettre à jour** des enregistrements existants selon des critères très précis.

- `UPDATE` : C'est le verbe de la requête. Il annonce l'intention de **modifier** des données existantes.
- `article` : C'est le nom de la table **dans laquelle** on veut effectuer les modification.
- `SET` : Ce mot-clé introduit la liste des **changements** à appliquer.
- `prix = 24.99` : "Affecte la valeur `24.99` à la colonne `prix`." L'ancienne valeur est écrasée.
- `stock = stock - 1` : "Prend la **valeur actuelle** de la colonne `stock`, soustrais 1, et sauvegarde le résultat dans la colonne `stock`." C'est une opération de calcul relative très courante (décrémentation).
- `WHERE` : C'est le mot-clé **le plus important** de cette requête. Il signifie **"où"** et introduit une condition qui **filtre** les lignes à modifier.
- `nom = 'Apprendre SQL'` : C'est la condition de filtrage. Elle dit : "N'applique les modifications **que** aux lignes où la valeur de la colonne `nom` est **exactement** égale à la chaîne de caractères 'Apprendre SQL'."

En résumé : La requête `UPDATE...SET...WHERE` est l'outil pour **corriger** ou **faire évoluer** les données existantes. La clause `WHERE` est le **gardien** qui garantit que seules les données que vous voulez vraiment changer le seront. Son oubli est l'une des erreurs les plus courantes et les plus graves en SQL.

3. Suppression de données (DELETE)

Supprimons tous les articles en rupture de stock.

```sql
DELETE FROM article
WHERE stock = 0; -- WHERE est tout aussi crucial ici !
```

Image 4 Requête de suppression

La requête `DELETE FROM article WHERE stock = 0;` signifie :

"Supprime de la table 'article' toutes les lignes où la valeur de la colonne 'stock' est égale à zéro."

En résumé, elle permet de **nettoyer la base de données** en supprimant définitivement les articles qui sont en rupture de stock.

- `DELETE FROM` : C'est le verbe de la requête. Il annonce l'intention de **supprimer définitivement** des lignes de données.
- `article` : C'est le nom de la table **de laquelle** on veut supprimer des lignes.
- `WHERE` : C'est le mot-clé **le plus important et le plus critique** de cette requête. Il signifie **"où"** et introduit une condition qui **filtre** les lignes à supprimer.
- `stock = 0` : C'est la condition de filtrage. Elle dit : "Ne supprime **que** les lignes où la valeur de la colonne `stock` est **exactement** égale à `0`."

⚠️ **Attention : `DELETE FROM table_name` sans `WHERE` supprime TOUTES les lignes de la table.**

Pourquoi la Clause `WHERE` est-elle "tout aussi cruciale ici" ?

Sans la clause `WHERE`, la requête aurait été :

```sql
DELETE FROM article;
```

Image 5 Exemple de suppression sans where

C**ette requête est CATASTROPHIQUE et IRREVERSIBLE.** Elle signifie :

"Supprime TOUTES les lignes de la table 'article'."

Résultat :

La table `article` serait **complètement vide**. Toutes vos données seraient perdues.

C'est la pire erreur possible en SQL. C'est pourquoi on insiste autant sur la clause `WHERE` pour les commandes `DELETE` (et `UPDATE`).

⚠️ **Règle d'or : TOUJOURS écrire la clause `WHERE` en premier dans une requête `DELETE` (et `UPDATE`) pour éviter de l'oublier.**

Bonnes Pratiques et Alternatives

1. **Vérification avant suppression :** Avant d'exécuter un `DELETE`, faites d'abord un `SELECT` avec la même condition `WHERE` pour voir quelles lignes seront affectées.

```sql
-- D'abord, vérifiez ce qui sera supprimé
SELECT * FROM article WHERE stock = 0;
-- Ensuite, si le résultat est correct, exécutez la suppression
DELETE FROM article WHERE stock = 0;
```

Image 6 Procédure de suppression

S**uppression logique vs physique :** Dans de nombreuses applications, on évite de supprimer physiquement des données. À la place, on fait une "suppression logique" en ajoutant une colonne comme `est_actif BOOLEAN DEFAULT TRUE`. Pour "supprimer" un article, on fait plutôt :

```sql
UPDATE article SET est_actif = FALSE WHERE stock = 0;
```

Image 7 Méthode de suppression

Cela permet de conserver l'historique des données tout en les masquant de l'application.

Quand Utiliser Cette Requête ?

- **Nettoyage de base de données :** Pour supprimer définitivement des données obsolètes ou erronées.
- **Conformité RGPD :** Pour supprimer des informations personnelles à la demande d'un utilisateur (`DELETE FROM clients WHERE id = 123;`).
- **Gestion de vieux logs :** Supprimer des entrées de journal trop anciennes.

En résumé : La requête `DELETE FROM ... WHERE ...` est un outil de **nettoyage puissant et définitif**. La clause `WHERE` est le **gardien indispensable** qui garantit que seules les données que vous voulez vraiment supprimer le seront. Son utilisation demande une extrême prudence.

Langage d'Interrogation de Données (LID - Data Query Language)



Le LID est le cœur de SQL. Il permet de **lire et d'extraire** des données des tables. La commande principale est `SELECT`.

Structure de base :

```sql
SELECT colonne1, colonne2 -- Quoi ?
FROM ma_table -- De où ?
WHERE condition; -- Sous quelles conditions ?
```

Image 8 Syntaxe de base pour l'interrogation de données

1. Sélection simple

Requête pour récupérer toutes les données d'une table :

```sql
SELECT * FROM article;
```

Image 9 Requête pour récupérer toutes les données de la table article

Décomposons chaque partie de la requête :

1. **`SELECT`** : "Sélectionne". C'est le verbe, l'action principale de la requête. Elle indique que vous voulez **récupérer** des données.
2. **`*`** (astérisque) :
   - **Signifie** "Tout" ou "Tous". Ici, il signifie **"toutes les colonnes"**.
   - Dans notre exemple de table `article` (avec les colonnes `id`, `nom`, `description`, `prix`, `stock`), `*` équivaut à écrire `id, nom, description, prix, stock`.
3. **`FROM`** : "De". Ce mot-clé introduit l'endroit d'où les données doivent être récupérées, c'est-à-dire le nom de la **table**.
4. **`article`** : C'est le nom de la **table** sur laquelle on exécute la requête. C'est la source des données.
5. `;` (point-virgule) : Il marque la **fin de l'instruction SQL**. C'est comme un point à la fin d'une phrase. Bien que parfois optionnel dans certains outils, il est considéré comme une bonne pratique de toujours le mettre.

Récupérons seulement les noms et prix des articles.

```sql
SELECT nom, prix FROM article;
```

Image 10 Requête pour récupérer les données d'une ou des colonnes spécifiques

2. Filtrage avec WHERE

Requête pour récupérer les articles dont le prix est supérieur à 20€.

```sql
SELECT * FROM article WHERE prix > 20;
```

Image 11

Décomposons chaque partie de la requête :

1. **`SELECT *`** :
    - **"Sélectionne toutes les colonnes"**. L'astérisque `*` est un raccourci pour dire "id, nom, description, prix, stock, etc." – bref, toutes les colonnes de la table.
2. **`FROM article`** :
    - **"De la table 'article'"**. Cela spécifie la source des données, la table sur laquelle on travaille.
3. **`WHERE`** :
    - **"Où"**. C'est le mot-clé le plus important ici. Il introduit une **condition de filtrage**. Il agit comme un tamis : il ne laissera passer que les lignes qui respectent la condition qui le suit.
4. **`prix > 20`** :
    - **"Le prix est supérieur à 20"**. C'est la **condition de filtrage** elle-même.
    - `prix` : Le nom de la colonne sur laquelle on filtre.
    - `>` : L'**opérateur de comparaison** "supérieur à".
    - `20` : La **valeur de référence** avec laquelle on compare chaque valeur de la colonne `prix`.
5. `;` : Marque la fin de l'instruction SQL.

- **Pourquoi et Quand Utiliser Cette Requête ?**
    - **Filtrer des résultats :** C'est son usage principal. Vous ne voulez pas voir l'intégralité des données, mais seulement un sous-ensemble qui vous intéresse.
    - **Analyses ciblées :** Par exemple, "quels sont nos produits haut de gamme ?", "quels articles sont en rupture de stock ?", etc.
    - **Fonctionnalités d'application :** C'est la base des filtres sur un site e-commerce ("montrer les produits entre 20€ et 50€").

  Variantes et Opérateurs Utiles avec `WHERE`

  Vous pouvez utiliser beaucoup d'autres opérateurs de comparaison :

    - **`=`** : Égal à. `WHERE prix = 19.99`
    - **`<`** : Inférieur à. `WHERE prix < 30`
    - **`>=`** : Supérieur ou égal à. `WHERE stock >= 5` (articles avec au moins 5 unités en stock)
    - **`<=`** : Inférieur ou égal à.
    - **`<>`** ou **`!=`** : Différent de. `WHERE categorie != 'livre'`
    - **`BETWEEN`** : Entre deux valeurs. `WHERE prix BETWEEN 15 AND 25`
    - **`LIKE`** : Pour chercher un motif textuel. `WHERE nom LIKE '%SQL%'` (nom qui contient "SQL")
    - **`IN`** : Pour une liste de valeurs. `WHERE id IN (1, 3, 5)`

  En résumé : `SELECT * FROM article WHERE prix > 20;` est la combinaison fondamentale pour **sélectionner** des données spécifiques selon une **condition**. C'est l'une des clauses les plus utilisées en SQL.

Requête pour récupérer les articles qui sont en stock (stock > 0) **ET** dont le prix est inférieur à 50€.

```sql
SELECT * FROM article WHERE stock > 0 AND prix < 50;
```

Image 12

Requête pour récupérer les articles dont le nom contient "SQL" (utilisation de `LIKE`)

```sql
SELECT * FROM article WHERE nom LIKE '%SQL%';
```

Image 13

3. Tri des résultats avec ORDER BY

Ordonner le résultat du moins cher au plus cher :

```sql
SELECT * FROM article ORDER BY prix ASC;
```

Image 14

Ordonner le résultat du plus cher au moins cher :

```sql
SELECT * FROM article ORDER BY prix DESC;
```

Image 15

4. Agrégation et regroupement avec GROUP BY

Fonctions courantes : `COUNT()`, `SUM()`, `AVG()`, `MAX()`, `MIN()`.

Nombre total d'articles en base :

"Compte le nombre total de lignes (d'articles) dans la table 'article' et retourne ce résultat dans une colonne temporaire que tu appelleras 'nombre_total_articles'."

En résumé, elle répond à la question : **"Combien y a-t-il d'articles au total dans mon catalogue ?"**

```sql
SELECT COUNT(*) AS nombre_total_articles FROM article;
```

Image 16

Décomposons chaque partie de la requête :

1. **`SELECT`** :
    - **"Sélectionne"**. Le verbe de la requête. Ici, il est utilisé pour demander le retour d'une valeur calculée (le compte).
2. **`COUNT(*)`** :
    - C'est la **fonction d'agrégation**, le cœur de la requête.
    - **`COUNT()`** : Une fonction qui **compte** le nombre de lignes.
    - **`*`** : À l'intérieur de la fonction `COUNT()`, l'astérisque signifie **"compter toutes les lignes, sans exception"**, peu importe si des colonnes contiennent des valeurs `NULL` ou vides. Elle compte chaque ligne existante.
3. **`AS nombre_total_articles`** :
    - **`AS`** : Signifie **"en tant que"** ou **"alias"**. C'est un mot-clé qui permet de **donner un nom personnalisé** à la colonne qui contient le résultat du calcul.
    - **`nombre_total_articles`** : Le nom de l'alias, le label que vous choisissez pour le résultat. Sans `AS`, la colonne de résultat s'appellerait juste `COUNT(*)`, ce qui est peu lisible.
4. **`FROM article`** :
    - **"De la table 'article'"**. Cela spécifie la table sur laquelle la fonction `COUNT(*)` doit opérer. C'est là que le moteur SQL va compter les lignes.
5. `;` : Marque la fin de l'instruction.

Pourquoi et Quand Utiliser Cette Requête ?

- **Obtention de métriques :** C'est la requête par excellence pour obtenir des statistiques simples mais cruciales : nombre total d'utilisateurs, de commandes, de produits, etc.
- **Vérification :** Après une importation massive de données, vous pouvez l'utiliser pour vérifier que le nombre d'enregistrements correspond à ce qui était attendu.

- **Dans des requêtes plus complexes :** Elle est souvent utilisée dans des sous-requêtes ou avec `GROUP BY` pour compter des éléments par catégorie (ex: le nombre d'articles par catégorie).

Variantes et Subtilités

- **`COUNT(*)` vs `COUNT(nom_de_colonne)`** :
  - `COUNT(*)` compte **toutes les lignes**, même si toutes les colonnes sont `NULL` (ce qui est impossible avec une clé primaire).
  - `COUNT(description)` compterait seulement les lignes où la colonne `description` **n'est pas `NULL`**. Si un article n'a pas de description, il ne serait pas compté. C'est utile pour savoir combien de valeurs renseignées vous avez dans une colonne spécifique.
- **Utilisation avec `WHERE`** :

  Vous pouvez bien sûr combiner `COUNT(*)` avec une clause `WHERE` pour compter des sous-ensembles.

  Exemple : Compter le nombre d'articles dont le prix est supérieur à 20€.

```sql
SELECT COUNT(*) AS nb_articles_chers
FROM article
WHERE prix > 20;
```

Image 5

Prix moyen des articles par catégorie.

Pour chaque catégorie, calcule le prix moyen des articles qui lui sont associés. Affiche le nom de la catégorie et ce prix moyen."

En résumé, elle répond à une question business cruciale : **"Quel est le prix moyen d'un article dans chaque catégorie de mon catalogue ?"** (Ex: le prix moyen des "Livres" est de 22€, celui des "Vêtements" est de 45€, etc.)

```sql
SELECT c.nom, AVG(a.prix) AS prix_moyen
FROM article a
JOIN categorie c ON a.id_categorie = c.id
GROUP BY c.nom;
```

Image 6

- **`c.nom`** : Ici, on sélectionne la colonne `nom` de la table `categorie`. Le `c.` est un **alias** qui précise de quelle table vient la colonne (voir point 2).
- **`AVG(a.prix) AS prix_moyen`** :
  - `AVG()` : C'est une **fonction d'agrégation**. Elle calcule la moyenne arithmétique de toutes les valeurs dans la colonne `prix`.
  - `a.prix` : On précise qu'on veut la moyenne de la colonne `prix` de la table `article`.
  - `AS prix_moyen` : Donne un nom clair (`prix_moyen`) à la colonne qui contiendra le résultat du calcul. Sans cela, elle s'appellerait `AVG(a.prix)`, ce qui est illisible.
- **`FROM article a`** : On dit "va chercher les données dans la table `article`". On lui donne un **alias** (un surnom) : `a`. Cela permet d'écrire `a.prix` au lieu de `article.prix`.
- **`JOIN categorie c`** : "Associe-la (**JOIN**) avec la table `categorie`", à qui on donne l'alias `c`.
- **`ON a.id_categorie = c.id`** : "Fais cette association **sur la condition** que l'`id_categorie` dans la table `article` corresponde à l'`id` dans la table `categorie`."
  - C'est ce qui permet de "lier" un article à sa catégorie correcte.

5. Jointures (JOIN)

C'est la clé des bases relationnelles ! Cela permet de combiner des lignes de plusieurs tables.

INNER JOIN : Affiche seulement les articles qui ont une catégorie associée.

"Pour chaque article, trouve sa catégorie correspondante et affiche-moi le nom de l'article, son prix, et le nom de sa catégorie."

En résumé, elle permet de **combiner des informations** qui sont stockées dans deux tables différentes mais qui sont liées entre elles, grâce à la clé étrangère `id_categorie`.

```sql
SELECT a.nom, a.prix, c.nom AS nom_categorie
FROM article a
INNER JOIN categorie c ON a.id_categorie = c.id;
```

Image 17

- **`a.nom`** : On sélectionne la colonne `nom` de la table `article`. Le `a.` est un **alias** qui précise de quelle table vient la colonne.
- **`a.prix`** : On sélectionne la colonne `prix` de la table `article`.
- **`c.nom AS nom_categorie`** :
  - `c.nom` : On sélectionne la colonne `nom` de la table `categorie`.
  - **`AS nom_categorie`** : C'est un **alias** (un renommage temporaire). Il est **crucial ici** car les deux tables ont une colonne `nom`. Pour éviter un conflit et rendre le résultat clair, on renomme la colonne `c.nom` en `nom_categorie` dans le résultat final.
- **`FROM article`** : On spécifie que la table principale de laquelle on part est `article`.
- **`a`** : On donne un **alias** (un surnom) à la table `article`. Cela nous permet d'écrire `a.nom` au lieu de `article.nom`, ce qui est plus court et plus lisible, surtout quand les noms de tables sont longs.
- **`INNER JOIN categorie c`** : "**Associe** (**JOIN**) la table `categorie` de manière **interne** (**INNER**)".
  - `c` : On donne aussi un alias à la table `categorie`.
- **`ON a.id_categorie = c.id`** : "**Sur la condition** que la valeur de la colonne `id_categorie` dans la table `article` soit **égale à** la valeur de la colonne `id` dans la table `categorie`."
  - Cette condition est le **lien logique** entre les deux tables. C'est grâce à elle que l'on peut "retrouver" la catégorie de chaque article.

LEFT JOIN : Affiche tous les articles, même ceux sans catégorie :

```sql
SELECT a.nom, a.prix, c.nom AS nom_categorie
FROM article a
LEFT JOIN categorie c ON a.id_categorie = c.id;
```

Image 18

Langage de Contrôle de Données (LCD - Data Control Language)



Le LCD permet de **gérer les permissions** des utilisateurs sur la base de données.

Commandes clés : `GRANT`, `REVOKE`

Note importante : SQLite offre une gestion des permissions très basique qui est souvent gérée par le système de fichiers du OS. Pour des systèmes complets (MySQL, PostgreSQL), le LCD est essentiel.

Concepts (illustratifs) :

- Donner à un utilisateur "lecteur" le droit de faire des `SELECT` sur la table `article` :

```
GRANT SELECT ON article TO lecteur;
```

Image 19 Requête donnant le droit à un utilisateur d'exécuter des requêtes select sur la table article

Cette requête donne à l'utilisateur 'lecteur' la permission de lire les données de la table 'article', mais pas de les modifier, de les supprimer ou d'en ajouter.

En résumé, c'est une commande de **contrôle d'accès** qui permet de définir qui peut faire quoi dans la base de données.

- **`GRANT`** : C'est le verbe de la requête. Il signifie **"accorder"** ou **"donner"** une permission. C'est l'opposé de `REVOKE` qui signifie "révoquer".
- **`SELECT`** : C'est le **privilège** spécifique que l'on accorde. Il autorise l'utilisateur à **lire** les données (exécuter des requêtes `SELECT`) sur la table concernée.

D'autres privilèges courants existent :

- `INSERT` : Permet d'ajouter de nouvelles lignes.
- `UPDATE` : Permet de modifier des lignes existantes.
- `DELETE` : Permet de supprimer des lignes.
- `ALL` ou `ALL PRIVILEGES` : Donne tous les droits possibles sur la table.
- **`ON`** : Précise **l'objet** sur lequel le privilège est accordé.
- **`article`** : Le nom de la **table** spécifique sur laquelle on donne l'autorisation. On pourrait aussi accorder des droits sur une vue (`VIEW`) ou même toute la base (`ON DATABASE ...`).
- **`TO`** : Introduit le destinataire du privilège.
- **`lecteur`** : Le nom de l'**utilisateur** (ou du **rôle**) auquel on accorde le droit. Un "rôle" est un groupe d'utilisateurs auquel on peut attribuer des permissions (ex: `lecteur`, `editeur`, `administrateur`).

Importance de la Sécurité par les Permissions

Cette commande est cruciale pour plusieurs raisons :

1. **Principe du Moindre Privilège :** C'est une règle de sécurité fondamentale. On ne donne à un utilisateur que les permissions strictement nécessaires pour accomplir sa tâche. Un compte pour une application web n'a souvent besoin que de `SELECT`, `INSERT`, `UPDATE`, mais presque jamais de `DELETE` ou de `DROP TABLE`.

2. **Séparation des Tâches :**
    - Un analyste data a besoin de `SELECT` sur beaucoup de tables.
    - Un application de front-end a besoin de `SELECT`, `INSERT`, `UPDATE` sur les tables liées aux utilisateurs et aux commandes.
    - Un administrateur a besoin de `ALL PRIVILEGES`.
3. **Protection contre les Erreurs et les Attaques :** Si un pirate arrive à voler les identifiants de l'utilisateur 'lecteur', il ne pourra pas modifier ou supprimer vos données, limitant ainsi les dégâts.

En résumé : La requête `GRANT SELECT ON article TO lecteur;` est une commande de **sécurité** qui permet de configurer finement les autorisations des utilisateurs dans une base de données. Elle est essentielle pour toute application professionnelle afin de garantir la confidentialité, l'intégrité et la disponibilité des données.

Donner à un utilisateur "editeur" tous les droits sur la table `article` :

```
GRANT SELECT, INSERT, UPDATE, DELETE ON article TO editeur;
```

Image 2

Retirer le droit `DELETE` à l'utilisateur "editeur" :

```
REVOKE DELETE ON article FROM editeur;
```

Image 3

Dans SQLite, la sécurité se gère souvent par le chiffrement de la base de données ou les permissions d'accès au fichier `.db`.

Conclusion

Récapitulons les points clés de cette leçon :

- **LDD (`CREATE`, `ALTER`, `DROP`)** : Vous savez maintenant définir la structure de vos données.
- **LID (`SELECT`, `WHERE`, `JOIN`, `GROUP BY`)** : Vous maîtrisez l'art d'interroger la base pour extraire des informations précieuses et significatives.
- **LMD (`INSERT`, `UPDATE`, `DELETE`)** : Vous pouvez manipuler les données en toute sécurité, en n'oubliant jamais la clause `WHERE` !
- **LCD (`GRANT`, `REVOKE`)** : Vous avez compris l'importance de contrôler l'accès aux données pour la sécurité.

Pour aller plus loin :

- **Pratiquez !** La pratique est la clé. Utilisez des outils comme **DB Browser for SQLite** (une interface graphique) ou des sites comme **SQLZoo**, **LeetCode** ou **HackerRank** pour vous exercer sur des problèmes variés.
- **Explorez les fonctions SQLite** : Date/Time, JSON, les Common Table Expressions (CTE avec `WITH`).
- **Intégration** : Apprenez à utiliser SQLite avec un langage de programmation (Python avec la bibliothèque `sqlite3`, JavaScript avec Node.js, etc.).

Félicitations ! Vous avez acquis les fondations solides de SQL. Continuez à pratiquer pour consolider ces connaissances.